

МОСКОВСКИЙ ФИЗИКО–ТЕХНИЧЕСКИЙ ИНСТИТУТ

АРХИТЕКТУРА ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ
ЗАДАНИЕ №3



ФИО: Лазарь Владислав Игоревич

Группа: Б01-207а

Почта: lazar.v@phystech.edu

Вариант №3: bpu_v3.asm

Долгопрудный, 2026 г.

Часть 1.

Запустим симуляцию программы `./bpu_v3.asm` в **RARS Tool** с подключенным **BHT Simulator**. Используем следующие настройки **BHT**:

- # of BHT entries: 16
- BHT history size: 1
- Initial value: NOT TAKE

Branch History Table Simulator						
# of BHT entries		16	BHT history size	1	Initial value	NOT TAKE
Instruction		Index	History	Prediction	Correct	Incorrect
bne x11,x5,-20		0	NT	NOT TAKE	0	0
@ Address		1	NT	NOT TAKE	0	0
0x400028		2	NT	NOT TAKE	0	0
-> Index		3	NT	NOT TAKE	0	0
10		4	NT	NOT TAKE	0	0
		5	T	TAKE	9999	1
		6	NT	NOT TAKE	0	0
		7	NT	NOT TAKE	10000	0
		8	NT	NOT TAKE	0	0
		9	NT	NOT TAKE	0	0
		10	NT	NOT TAKE	9998	2
		11	NT	NOT TAKE	0	0
		12	NT	NOT TAKE	0	0
		13	NT	NOT TAKE	0	0
		14	NT	NOT TAKE	0	0
		15	NT	NOT TAKE	0	0

Рис. 1: Состояние **BHT** в конце симуляции программы

Определим точность предсказаний для инструкций перехода:

- Для `magic_br_1` имеем:

Эта инструкция проверяет `a4`. В коде имеем `a4 = 0` и условие `beq a4, zero, magic_br_1`. При первой итерации предсказатель (`Initial value: NOT TAKE`) ошибся. После этого он переключился в состояние **T (TAKEN)** и был прав остальные 9999 раз. Точность для `magic_br_1` равна 99.99%.

- Для `magic_br_2` имеем:

Эта инструкция проверяет `a3`. В коде имеем `a3 = 1` и условие `beq a3, zero, magic_br_2`. Так как начальное значение предсказателя было `NOT TAKE`, и переход действительно ни разу не случился, предсказатель угадал в 100% случаев.

Часть 2.

Добавим теперь в ассемблерный код инструкции `nor` таким образом, чтобы точность предсказаний для обеих инструкций прямого условного перехода `magic_br_1` и `magic_br_2` составила ровно 0. Настройки для **ВНТ** будут теми же, что и в первой части.

Таблица **ВНТ** имеет 16 записей. Индекс в таблице вычисляется на основе младших битов адреса инструкции. Чтобы инструкции `magic_br_1` и `magic_br_2` использовали одну и ту же ячейку **ВНТ**, разница между их адресами должна быть кратна $16 \times 4 = 64$ байтам.

Branch History Table Simulator						
# of BHT entries		16	BHT history size	1	Initial value	NOT TAKE
Instruction	Index	History	Prediction	Correct	Incorrect	Precision
bne x11,x5,-76	0	NT	NOT TAKE	0	0	0,00
@ Address	1	NT	NOT TAKE	0	0	0,00
	2	NT	NOT TAKE	0	0	0,00
	3	NT	NOT TAKE	0	0	0,00
	4	NT	NOT TAKE	0	0	0,00
	5	NT	NOT TAKE	0	20000	0,00
	6	NT	NOT TAKE	0	0	0,00
	7	NT	NOT TAKE	0	0	0,00
	8	NT	NOT TAKE	9998	2	99,98
	9	NT	NOT TAKE	0	0	0,00
	10	NT	NOT TAKE	0	0	0,00
	11	NT	NOT TAKE	0	0	0,00
	12	NT	NOT TAKE	0	0	0,00
	13	NT	NOT TAKE	0	0	0,00
	14	NT	NOT TAKE	0	0	0,00
	15	NT	NOT TAKE	0	0	0,00
-> Index						
		8				

Рис. 2: Состояние **ВНТ** в конце симуляции программы

Реализацию этой части можно посмотреть в файле `./task2.asm`.

Между инструкциями ветвления было вставлено 14 инструкций `nor` (с учетом инструкций `addi` и `beq`). Теперь оба перехода имеют одинаковый индекс в **ВНТ**.

В каждой итерации цикла `magic_br_1` записывает в общую ячейку **ВНТ** значение `T`, из-за чего следующий за ним `magic_br_2` предсказывается неверно и перезаписывает ячейку на `NT` (`Not Taken`). В следующем цикле `magic_br_1` снова ошибается из-за значения `NT`, оставленного предыдущей командой. Таким образом, предсказатель ошибается на каждом шаге для обеих инструкций, что дает точность 0%.

Часть 3.

Запустим симуляцию модифицированной программы (из второй части), используя новые настройки **ВНТ**:

- # of BHT entries: 16
- BHT history size: 2
- Initial value: NOT TAKE

Branch History Table Simulator						
# of BHT entries		16	BHT history size	2	Initial value	NOT TAKE
Instruction		Index	History	Prediction	Correct	Incorrect
bne x11,x5,-76		0	NT, NT	NOT TAKE	0	0
@ Address		1	NT, NT	NOT TAKE	0	0
0x400060		2	NT, NT	NOT TAKE	0	0
-> Index		3	NT, NT	NOT TAKE	0	0
8		4	NT, NT	NOT TAKE	0	0
		5	T, NT	NOT TAKE	10000	10000
		6	NT, NT	NOT TAKE	0	0
		7	NT, NT	NOT TAKE	0	0
		8	T, NT	TAKE	9997	3
		9	NT, NT	NOT TAKE	0	0
		10	NT, NT	NOT TAKE	0	0
		11	NT, NT	NOT TAKE	0	0
		12	NT, NT	NOT TAKE	0	0
		13	NT, NT	NOT TAKE	0	0
		14	NT, NT	NOT TAKE	0	0
		15	NT, NT	NOT TAKE	0	0

Рис. 3: Состояние **ВНТ** в конце симуляции программы

Определим точность предсказаний для инструкций перехода. Мы уже знаем, что для `magic_br_1` — всегда T, `magic_br_2` — всегда NT.

Процесс изменения состояний в таблице:

1. В начале будет состояние NT, NT.
2. `magic_br_1` (T): в предсказателе имеем NT, что является ошибкой. Состояние меняется на T, NT.
3. `magic_br_2` (NT): в предсказателе имеем NT, что является верным предсказанием. Состояние возвращается в NT, NT.
4. Далее итеративно `magic_br_1` (T) предсказывается как NT — ошибка, и `magic_br_1` (T) как NT — верное предсказание.

Итого 100% попаданий для одной ветки и 0% для другой дают среднее значение 50%.

Часть 4.

Дополним код программы из пункта 2. Добавим еще 2 инструкции прямого условного перехода `beq` так, чтобы точность предсказаний для всех 4 инструкций прямого условного перехода (`magic_br_1`, `magic_br_2` + 2 новых инструкции) не превышала 0.0001.

Причем, можно использовать не только инструкции `beq`. Настройки для **ВНТ** будут теми же, что и в пункте 3:

- # of BHT entries: 16
- BHT history size: 2
- Initial value: NOT TAKE

Branch History Table Simulator						
# of BHT entries		16	BHT history size		2	Initial value
						NOT TAKE
Instruction	Index	History	Prediction	Correct	Incorrect	Precision
bne x11,x5,-204 @ Address 0x4000e0 -> Index 8	0	NT, NT	NOT TAKE	0	0	0,00
	1	NT, NT	NOT TAKE	0	0	0,00
	2	NT, NT	NOT TAKE	0	0	0,00
	3	NT, NT	NOT TAKE	0	0	0,00
	4	NT, NT	NOT TAKE	0	0	0,00
	5	NT, T	NOT TAKE	2	39998	0,01
	6	NT, NT	NOT TAKE	0	0	0,00
	7	NT, NT	NOT TAKE	0	0	0,00
	8	T, NT	TAKE	9997	3	99,97
	9	NT, NT	NOT TAKE	0	0	0,00
	10	NT, NT	NOT TAKE	0	0	0,00
	11	NT, NT	NOT TAKE	0	0	0,00
	12	NT, NT	NOT TAKE	0	0	0,00
	13	NT, NT	NOT TAKE	0	0	0,00
	14	NT, NT	NOT TAKE	0	0	0,00
	15	NT, NT	NOT TAKE	0	0	0,00

Рис. 4: Состояние **ВНТ** в конце симуляции программы

Реализацию этой части можно посмотреть в файле `./task4.asm`.

Здесь мы воспользовались идеей из второго пункта, вставив `por` таким образом, чтобы все 4 инструкции ветвления отображались в один и тот же индекс таблицы **ВНТ**.

Процесс изменения состояний в таблице:

1. `magic_br_1` (T): предсказатель выдает NT, что является ошибкой. Состояние счетчика инкрементируется и переходит в WNT (Weakly Not Taken).
2. `magic_br_2` (NT): предсказатель выдает NT, что является верным предсказанием. Состояние счетчика декрементируется и возвращается в SNT (Strongly Not Taken).
3. `magic_br_3` (NT): предсказатель выдает NT, что является верным предсказанием. Состояние подтверждается как SNT.
4. `magic_br_4` (T): предсказатель выдает NT, что является ошибкой. Состояние вновь переходит в WNT.

5. Далее итеративно в начале следующей итерации цикла для первой ветки `magic_br_1` предсказатель предложит NT, основываясь на состоянии WNT. Поскольку реальный исход — T, это снова приведет к ошибке.

В результате такой последовательности $\{T, NT, NT, T\}$ в рамках одного индекса, бимодальный предсказатель как бы "застревает" в состояниях SNT и WNT. Каждая ветка, требующая перехода (T), сталкивается с предсказанием NT, а ветки NT лишь подтверждают ошибочный для следующего шага развитие, что в совокупности удерживает точность на уровне $\approx 0\%$.